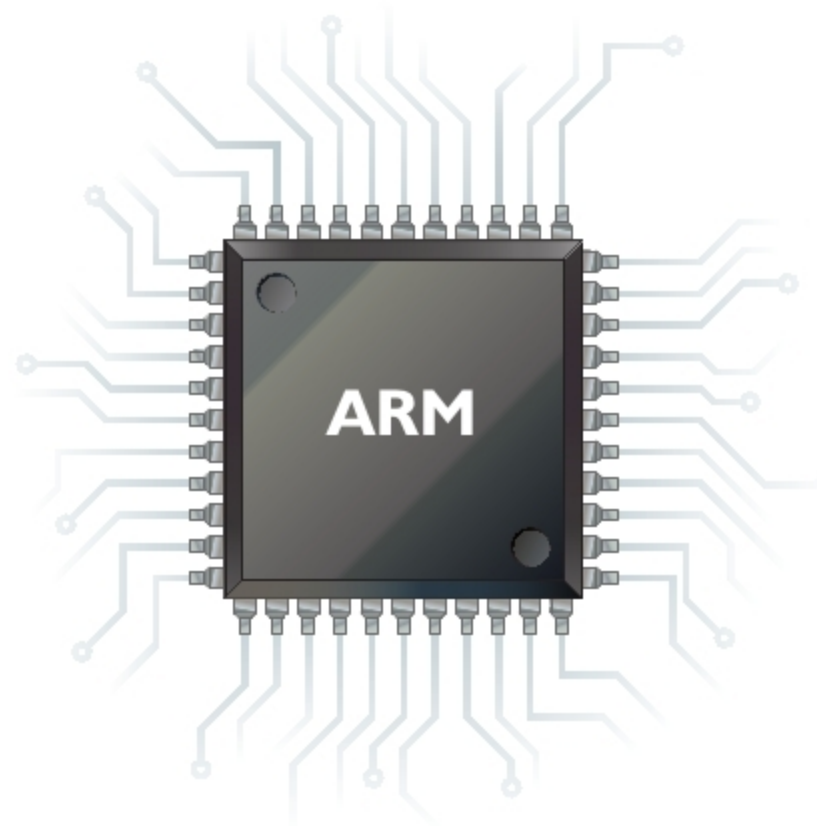


Creating an ARM Architecture Environment **Inside Linux**

The use of ARM processors is widespread in mobile phones, with around 80 per cent of today's mobiles having at least one ARM processor. The ARM environment can also be created in Linux machines through a virtual machine using chroot, QEMU and Linaro. Let's see how...



If you want to get your hands on ARM architecture or are interested in debugging ARM code, then you probably would be wondering how you could create an ARM environment (or use the ARM toolset) inside an Intel x86 or x64 architecture. This article will explain how this can be achieved by using QEMU, chroot and Linaro.

Almost 80 per cent of all mobile devices we use today have at least one ARM processor as part of the hardware. The ARM processor is a CPU built on the RISC-based architecture developed by Acorn Computers in the 1980s, and is now developed by Advanced RISC Machines (thus ARM).

The use of the ARM processor is very widespread and is not limited to mobile devices. It is also being heavily used as a processor for smart TVs, set-top boxes, IoT devices and most other embedded devices. If you develop applications for an ARM architecture-supporting OS or are learning ARM architecture fundamentals, the chances are that you will want to create a virtual environment of the ARM processor on your Linux system which is probably running on an Intel x86/x64 processor. Learning or being familiar with ARM is a great investment in your future; so without wasting more time, let's get started.

Creating a chroot environment for ARM

In general, chroot is only capable of creating virtual environments of the same architecture distributions. For

example, you can only create a Debian x86 system inside the x86 host system. So, to successfully create an ARM system, we need to use QEMU, which is capable of providing successful emulation for a number of ARM platforms.

Note: I am using Ubuntu 17.10(x64) as my host system. But all the steps remain the same for any other distro or architecture.

Installing the required packages

As mentioned earlier, we need to use QEMU for creating a cross-platform chroot environment. You can install QEMU and some other required packages using the following command:

```
$ apt-get install binfmt-support qemu qemu-user-static
```

binfmt-support is a kernel module that will help us execute the binary packages of the ARM architecture. If you want to be able to cross-compile packages using your host computer to build code that will run on the simulator, you'll need to install the GCC packages, as follows:

```
$ sudo apt-get install gcc-arm-linux-gnueabi g++-arm-linux-gnueabi
```